A login without the password



ATTACK PAYLOAD

```
{ "password": { "$ne": "" } }
```

One object changes the meaning of the login query.

Controlled experiment across five login configurations

Ercong Kee / Pheobe Neo / Kendrick Poon / Liu Qingyu

Ron et al., 2016 / controlled local reproduction

A query is a test on a record

DOCUMENT = A STORED RECORD

```
{ username: "alice", password: "…" }
```

Alice’s stored password is a non-empty string in the vulnerable lab.

EQUALITY CONDITION

```
password: "wrong"
```

Does the password equal “wrong”?

OPERATOR CONDITION

```
password: { $ne: "" }
```

Is the password different from an empty string?

Predicate = condition **$ne = not equal** **"" = empty string**

NoSQL models: document, key-value, graph, wide-column. Here: MongoDB documents.

MongoDB 7.0 $ne documentation / illustrative teaching record

Where the login goes wrong

THE TRUST BOUNDARY

01

Attacker controls the request

Knows "alice" and controls the JSON body

→

02 APPLICATION

Copies request values into the query

The client changes what the password check means.

→

03 MONGODB

Returns Alice

The query condition is true for the stored record.

A database match is not password proof.

Session created → **Alice remembered as the user** → **Private notes exposed**

Threat model: no direct database access and no modification of stored records.

The paper covers several attack boundaries

OUR MEASURED ROUTE

Query condition → **Operator object changes the condition** → **A different record can match**

Tautology-style operator injection in our experiment

OTHER BOUNDARIES DISCUSSED IN THE PAPER

Query text	JavaScript source	Protocol command	Trusted browser
Added logic changes matching records	Input becomes executable code	A separator adds a command	CSRF uses a user's authority

The rest of the talk tests which controls stop the operator route.

The password value becomes a condition

01 CLIENT REQUEST

username

```
"alice"
```

password

```
{ "$ne": "" }
```

The value is an object.



02 VULNERABLE APPLICATION

```
users.findOne({
  username: body.username,
  password: body.password
})
```

The app lets the request define the password condition.



03 DATABASE

username equals Alice

AND

password is not empty

MATCH

The client changed the meaning of the password check.

For this synthetic record, the stored password is non-empty. $ne: "" also matches missing fields.

An object and a string behave differently

OBJECT

```
{ "$ne": "" }
```

MongoDB sees

an operator condition

Alice matches

STRING

```
"{\"$ne\":\"\"}"
```

MongoDB sees

ordinary literal text

Alice does not match

Same characters. Different parsed types. Different query meaning.

How we tested the controls

15 CASES × **5** CONFIGURATIONS × **2** RUNS = **150** RECORDED RESULTS

FOR EVERY REQUEST, WE RECORDED

Login decision

Did the app create a session?

Protected access

Could the caller open Alice’s notes?

Database query count

Did MongoDB receive a find command?

Independent probes verify the control settings. Repeated runs show local repeatability.

15 cases × 5 configurations × 2 runs / results.json

Same attack, different login decision

FOUR DEMO CHECKS

01 Wrong password	02 Operator object	03 After the fix	04 Correct password
Login rejected	Attack succeeds	Same attack stopped	Login works

ATTACK SUCCEEDS

HTTP 200: Session created

2. Database predicate

```
{
  "username": "alice",
  "password": {
    "$ne": ""
  }
}
```

Database query: issued

3. Application decision

```
{
  "authenticated": true,
  "account": "alice"
}
```

Alice: private synthetic project notes

SAME ATTACK IS STOPPED

HTTP 400: Credential contract rejected

2. Database predicate

Rejected before filter construction

Database query: none

3. Application decision

```
{
  "authenticated": false,
  "account": null
}
```

Protected notes: access denied

Live lab: http://127.0.0.1:4400 Offline video: 90 seconds

Actual browser captures / local synthetic lab / 1 November 2026

Which controls stopped the attack?

3 controls still allow the attack

2 application controls stop it before any query

Configuration	Attack outcome	Database query	Correct login
Baseline	**Attack succeeds**	1 query sent	Login works
JavaScript disabled	**Attack succeeds**	1 query sent	Login works
Read-only DB account	**Attack succeeds**	1 query sent	Login works
String validation	**Attack stopped**	No query sent	Login works
Full hardened login	**Attack stopped**	No query sent	Login works

Scripting and read-only controls do not stop this read-only operator attack.

HTTP evidence: attack succeeds = 200; stopped before query = 400; correct login = 200.

Focal attack C04 / complete matrix in slides 17–18

A safe login keeps the password out of the query

ATTACK INPUT

```
password:
{ "$ne": "" }
```



TYPE GATE

Password must be text

STOPPED
No database query

LEGITIMATE LOGIN

The database query contains only the username.

```
users.findOne({ username })
```

01	02	03	04
Validate strings	**Find account**	**Verify hash**	**Create session**
Username and password are non-empty text	MongoDB receives username only	App checks password against stored hash	Only after a successful comparison

The submitted password never becomes a database condition.

Local source / OWASP NoSQL and password-storage guidance

Different injection types need different defences

MEASURED IN OUR LAB

Operator injection

Object contains $ne → MongoDB reads a query condition

Result

Different records match

Defence: accept text only and build the query in application code.

LITERATURE COMPARISON

JavaScript injection

Text enters JavaScript source → JavaScript engine executes it

Result

Program logic changes

Defence: never build code from user text; disable unused scripting.

Different interpreter, different defence. We measured the operator route only.

Literature comparison / MongoDB 7.0 $where documentation

What our experiment shows

1 **The attack succeeds**

Operator object logs the attacker in as Alice

Session created
HTTP 200

2 **The attack is stopped**

Object rejected before MongoDB receives a query

No query sent
HTTP 400

3 **Legitimate login still works**

Correct credentials retain protected access

Access granted
HTTP 200

Scope: one recorded stack, synthetic accounts and 15 request cases.

Recorded local experiment / results.json and results.csv

The application must keep control of the query

APPLICATION RULE

Choose the query structure.
Validate input values.
Verify the password.

NEXT STUDY

Can flexible search preserve customer separation?



User-selected filters must never override the tenant rule.

Recorded environment

Component	Recorded value
MongoDB	7.0.0
Node.js	v23.6.1
Express / native driver	5.1.0 / 6.20.0
Transport / parser	Loopback HTTP / JSON / express.json
Password verification	scrypt N=131072, r=8, p=1
Evidence	15 cases × 5 modes × 2 runs

Original experiment record. Fresh browser captures: 1 November 2026.

The vulnerable session decision

CLIENT VALUES ENTER THE QUERY

```
const filter = {
  username: req.body.username,
  password: req.body.password
};
const account = await users.findOne(filter);
```

```
if (account) createSession(account);
```

Condensed vulnerable excerpt. A match is incorrectly accepted as password proof.

The hardened session decision

1 Validate exactly two non-empty strings

```
if (!validContract(body)) return reject(400);
```

2 Look up by username only

```
const account = await users.findOne({ username });
```

3 Verify a salted password hash

```
const ok = await verifyPassword(password, storedHash);
```

4 Create a session after verification

```
if (account && ok) createSession(account);
```

Condensed code. Full source includes field/length checks and a dummy hash for unknown users.

Case results C01 to C08

Case	Request shape	Three vulnerable modes	Protected modes
C01	Valid credentials	Login succeeds	Login succeeds
C02	Wrong string password	Login rejected	Login rejected
C03	Unknown username	Login rejected	Login rejected
C04	Password operator object	Attack succeeds	Stopped before query
C05	Username operator with wrong password	Login rejected	Stopped before query
C06	Operator-looking literal string	Login rejected	Login rejected
C07	Null password	Login rejected	Stopped before query
C08	Array password	Login rejected	Stopped before query

Vulnerable modes: baseline, scripting disabled and read-only. Both runs agree.

HTTP codes are secondary evidence: 200 accepted, 401 login rejected, 400 stopped before query.

Case results C09 to C15

Case	Request shape	Three vulnerable modes	Protected modes
C09	Missing password	Login rejected	Stopped before query
C10	Numeric password	Login rejected	Stopped before query
C11	Extra field with valid credentials	Request accepted	Stopped before query
C12	Top-level array	Login rejected	Stopped before query
C13	Both fields contain operators	Attack succeeds	Stopped before query
C14	Empty password	Login rejected	Stopped before query
C15	Empty username	Login rejected	Stopped before query

Vulnerable modes: baseline, scripting disabled and read-only. Both runs agree.

HTTP codes are secondary evidence: 200 accepted, 401 login rejected, 400 stopped before query.

The controls were independently checked

Control	Independent observation
Read-only identity	Authenticated as labreader with the read role
Read permitted	Alice lookup succeeds
Write denied	MongoDB blocks the write (error 13)
Scripting enabled	The fixed $where probe returns Alice
Scripting disabled	The same probe fails as expected (error 6108304)

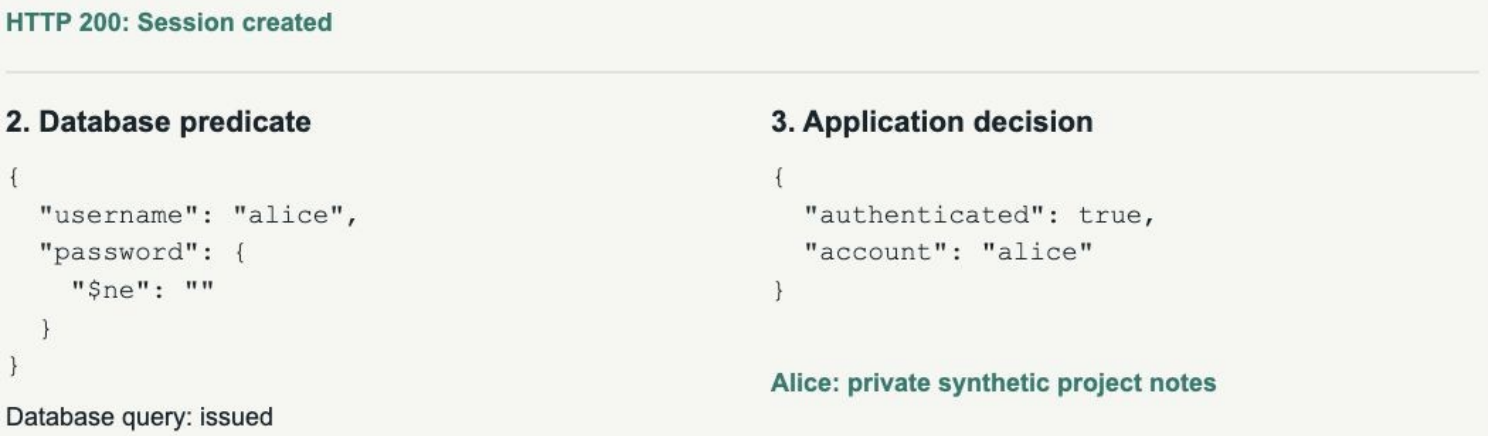
Both controls are active, but neither stops attack C04.

The fixed $where expression tests configuration, not attacker-controlled JavaScript.

Captured evidence: attack succeeds

ATTACK SUCCEEDS

```
{"username":"alice","password":{"$ne":""}}
```



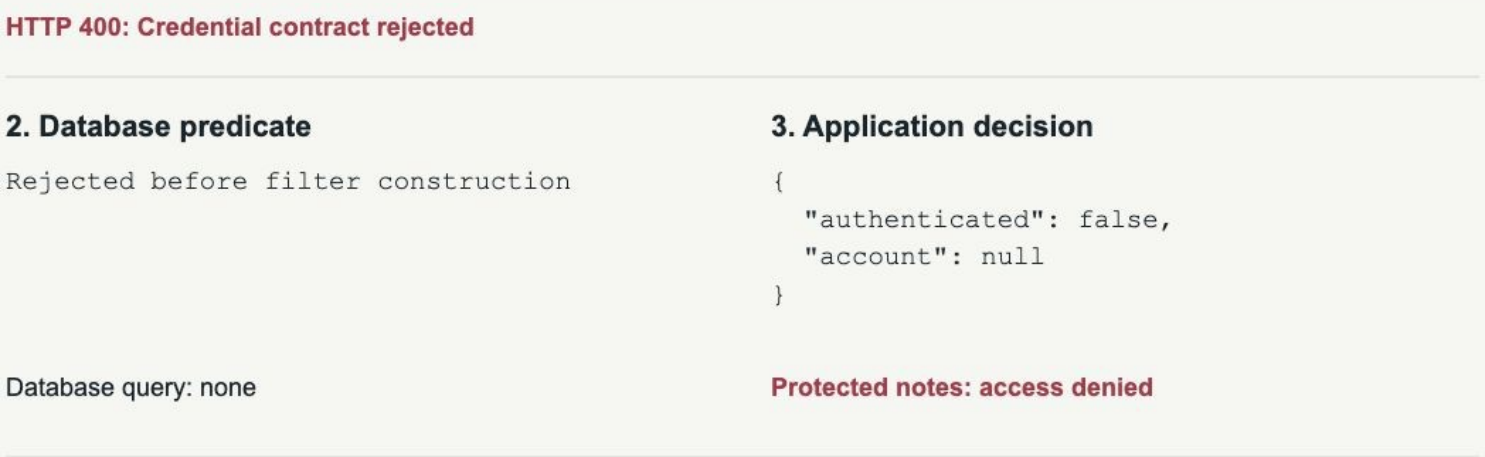
HTTP		
Session created		200
DATABASE		
Query sent		1 find
ACCESS		
Alice’s notes exposed		granted

The vulnerable app treats a record match as password proof.

Captured evidence: attack stopped

ATTACK STOPPED

```
{"username":"alice","password":{"$ne":""}}
```



HTTP

Credential contract rejected — 400

DATABASE

No query sent — 0 find

ACCESS

Protected notes denied — denied

The hardened app rejects the object before constructing a filter.

What remains untested

ESTABLISHED

This operator attack succeeds
Behaviour of five configurations
Rejection before a find query
Preservation of correct login

NOT ESTABLISHED

A JavaScript exploit
Real-world prevalence
All drivers or DB versions
Administrator access
Complete production security

Next experiment: permitted search without cross-tenant access.

Paper and database references

Ron, Shulman-Peleg and Puzanov (2016)
Analysis and Mitigation of NoSQL Injections
IEEE Security & Privacy 14(2), 30–39

DOI: 10.1109/MSP.2016.36

Author-uploaded manuscript and authors' InfoQ republication
Original classes: tautologies, union queries, JavaScript injection, piggybacked queries and origin violation.

MongoDB 7.0: $ne and $where documentation
Full URLs in the notes and team guide.

Defences and reproducible evidence

OWASP NoSQL Security Cheat Sheet
OWASP Password Storage Cheat Sheet

github.com/Qingyu255/CS440-Project

Evidence commit: c8252d2
Source and results are fixed at this revision.



results.json / results.csv / server.mjs / experiment.mjs
Fresh captures and team guide accompany the revised deck.